Gate 1 — How does a computer see?

Source: OpenCV-Python tutorials, "Getting Started with Images" and "Basic Operations on Images" (docs.opencv.org).

Q1. What is a digital image? Explain a pixel, image resolution, and the matrix representation of an image.

Explanation: Picture a sheet of graph paper with each tiny square coloured in. From far away it looks like a photo. A digital image works the same way.

Answer:

- A **digital image** is a grid of tiny coloured squares, stored as numbers.
- A **pixel** ("picture element") is one square in that grid. It holds one number for brightness, or three numbers for colour.
- **Resolution** is the size of the grid, written width × height. A 1920 × 1080 image has 1920 columns and 1080 rows, about 2 million pixels. Higher resolution means more detail but more data to process.
- **Matrix representation**: a matrix is a table of numbers with rows and columns. A grayscale image is one matrix. The value at row r, column c is that pixel's brightness, from 0 (black) to 255 (white). A colour image is three matrices stacked together, one each for red, green and blue. In Python, OpenCV gives you this as a NumPy array. img.shape returns something like (1080, 1920, 3), meaning rows, columns and colour channels.

Q2. What are RGB and grayscale images? Why might an autonomous vehicle process them differently?

Answer:

- An **RGB image** stores three values per pixel: Red, Green and Blue, each 0–255. Mixing them makes every colour. For example, (255, 0, 0) is pure red and (255, 255, 255) is white. Note that OpenCV stores them in the order **BGR**, not RGB, which surprises many beginners.
- A **grayscale image** stores one value per pixel, which is just brightness.
- **Why process them differently:** grayscale is one third the data, so it is faster. Edge detection mainly needs brightness changes, so grayscale is enough for it. Colour carries meaning that grayscale loses, though. A yellow lane line, a red traffic light or green grass can be told apart only in colour. So a vehicle often uses grayscale for fast edge-based steps and colour where the colour itself matters.

Gate 2 — Basic image processing

Sources: OpenCV tutorials "Smoothing Images", "Changing Colorspaces", "Canny Edge Detection"; PyImageSearch articles on smoothing/blurring and on cv2.Canny.

Q1. What is Gaussian blur and why is it useful before detecting features?

Explanation: Real photos have "noise": tiny random speckles, road grain and small texture. An edge detector looks for sudden brightness changes, so it treats every speckle as an edge.

Answer: Gaussian blur replaces each pixel with a weighted average of its neighbours. Nearby pixels count more and farther ones count less, following a bell-shaped (Gaussian) curve. This smooths out small noise while keeping large shapes like lane lines. The kernel size, for example 5 × 5, sets how many neighbours are averaged. A bigger kernel gives more blur. Blurring first means the edge detector finds real edges and fewer false ones.

Q2. What is HSV colour space? Why can it be more useful than RGB for lane detection?

Answer: HSV describes colour in a way closer to how people think about it:

- **H (Hue)**: which colour it is, such as red, yellow or blue. In OpenCV it ranges 0–179.
- **S (Saturation)**: how strong or pure the colour is. Low saturation looks greyish or whitish.
- **V (Value)**: how bright it is.

In RGB, brightness is mixed into all three numbers. A white line in shadow and the same line in sunlight have very different R, G and B values. In HSV, a shadow mainly changes V, while H and S stay similar. So you can pick out "white" (low S, high V) or "yellow" (H around 20–30) with simple ranges using cv2.inRange. That makes it more reliable under changing light.

Q3. What does Canny edge detection do?

Answer: Canny finds the outlines in an image, the places where brightness changes sharply. It works in four steps:

1. Blur the image to remove noise.
2. Measure how fast brightness changes at each pixel (the gradient) and in which direction.
3. Thin the edges to one pixel wide by keeping only the strongest point across each edge (non-maximum suppression).

4. Apply two thresholds (hysteresis). Pixels above the high threshold are definite edges. Pixels below the low threshold are dropped. Pixels in between are kept only if they connect to a definite edge.

The output is a black image with thin white lines where the edges are. In code: edges = cv2.Canny(gray, 50, 150).

Q4. What is a Region of Interest (ROI), and why can it reduce false detections?

Answer: An ROI is the part of the image you actually care about. For a forward-facing car camera, the lane is in the lower part of the image, roughly in a triangle or trapezium shape narrowing toward the horizon. You make a mask that is white inside that shape and black outside, then keep only the edges inside it. Edges from the sky, clouds, trees, grass and fences are thrown away before they can be mistaken for lane lines. Processing is also faster, because there is less to look at.

Gate 3 — Detecting structure

Sources: OpenCV tutorials "Contours: Getting Started" and "Hough Line Transform"; the "OpenCV Python Tutorial For Beginners" video series (lane detection episodes).

Q1. What is a contour? What information can a contour provide?

Answer: A contour is a curve that joins all the continuous boundary points of a shape that has the same colour or intensity. You can think of it as tracing around the outline of a blob in a black-and-white image. OpenCV finds them with cv2.findContours. From a contour you can measure:

- area (cv2.contourArea) and perimeter (cv2.arcLength)
- a bounding box around it (cv2.boundingRect)
- its centre point, using image moments
- its shape, such as long and thin (likely a lane marking) or roundish (likely something else)

This lets you filter objects, for example by keeping only long, thin white blobs as lane markings.

Q2. What is the Hough Transform? Explain how it can be used to find lane lines.

Explanation: After Canny you have a scatter of white edge pixels, but you do not yet know which ones form straight lines. The Hough Transform is a voting method that answers that question.

Answer:

1. Any straight line can be described by two numbers: ρ (rho), its distance from the image corner, and θ (theta), its angle.
2. Many different lines could pass through one edge pixel. That pixel casts one vote for every (ρ, θ) line it could lie on.
3. Every edge pixel votes the same way. Votes are counted in a table called the accumulator.
4. When many pixels lie on the same real line, that line's (ρ, θ) cell collects many votes. Cells with more votes than a threshold are reported as detected lines.

For lanes you normally use cv2.HoughLinesP (the "probabilistic" version). It returns line segments as end points (x1, y1, x2, y2). You then sort the segments by slope. Segments leaning one way belong to the left boundary and the other way to the right boundary. In image coordinates, the left lane line usually has a negative slope and the right one a positive slope. Average each group into one line and extend it to the bottom of the image and up toward the horizon.

Gate 4 — Lane / path perception

Q1. Why is lane/path detection useful for an autonomous ground vehicle?

Answer: It tells the vehicle where the drivable road is and where its own lane lies. From the lane, the vehicle can compute its centre line and how far it has drifted from it. That number feeds directly into the steering controller from PS 3. It also helps the vehicle stay legal and predictable, by not crossing into oncoming traffic, and it provides the local path for the planner.

Q2. Why can shadows, road texture, poor lighting, missing markings, or clutter create false detections?

Answer: Simple methods like Canny and thresholds only see brightness and colour changes. They do not understand what an object is.

- **Shadows** from trees or poles create strong dark–light edges that look like lines.
- **Road texture** such as cracks, tar patches and repairs creates edges too.
- **Poor lighting** at night, in glare or in rain reduces contrast, so real lines become faint, or reflections add false bright spots.
- **Missing or faded markings** leave nothing to detect, so the algorithm may lock onto a road edge or curb instead.
- **Clutter** such as other cars, arrows painted on the road and road signs adds extra edges inside the ROI.

Each of these breaks the assumption that the strongest straight edges are the lane lines.

Q3. Why is lane detection alone not sufficient for a fully autonomous vehicle?

Answer: Lanes only describe the road surface. A vehicle also needs to:

- detect obstacles, pedestrians, cyclists and other vehicles, and how they move
- read traffic lights and signs
- know its own position on a map (localisation)
- plan routes, decide when to change lanes or turn, and handle junctions where there are no lane lines
- cope when the camera fails, using other sensors such as LiDAR, radar and GPS

Lane detection is one input among many in the perception–planning–control chain.

Gate 5 — Perspective and BEV

Source: OpenCV tutorial "Geometric Transformations of Images", section "Perspective Transformation".

Q1. What is perspective distortion?

Answer: Things farther from the camera look smaller. Parallel lines appear to meet at a point on the horizon, called the vanishing point. Railway tracks are the classic example. The two lane lines are parallel in real life, but in the camera image they form a V or triangle. Distances are also uneven in the image: one pixel near the top covers many metres of road, while one pixel near the bottom covers a few centimetres. This makes measuring curves and distances directly from the image misleading.

Q2. What is a Bird's-Eye View (BEV)? Why can a top-down view make navigation perception easier?

Answer: A BEV is the image warped so it looks as if a camera were looking straight down at the road from above. You pick four points on the road in the camera image, such as the corners of a trapezium around the lane. You tell OpenCV where those points should go in a rectangle, using cv2.getPerspectiveTransform, and then warp the image with cv2.warpPerspective.

In BEV:

- lane lines become parallel again, which makes them easier to fit and check
- curvature can be measured properly
- distances are roughly uniform, so pixels convert to metres with one scale

- the picture matches a map or grid, which is exactly what the planner in PS 2 uses